

CAMPO

Documento técnico

Arquitectura, infraestructura y operación

Preparado para

Luis Mayo

FOSMON Construcciones · Mayo 2026

<https://campo-fosmon.netlify.app>

1. ¿Qué es CAMPO?

CAMPO es una **aplicación web tipo SPA (Single Page Application)** que reemplaza el flujo manual de Excel + WhatsApp para el control operativo y financiero de las obras de FOSMON. Funciona como una **PWA (Progressive Web App)** — se instala como app nativa en iPhone, Android y desktop.

Objetivo: centralizar avance físico, gastos, estimaciones, riesgo, subcontratos y documentación de cada obra en una sola plataforma multi-usuario con permisos por rol.

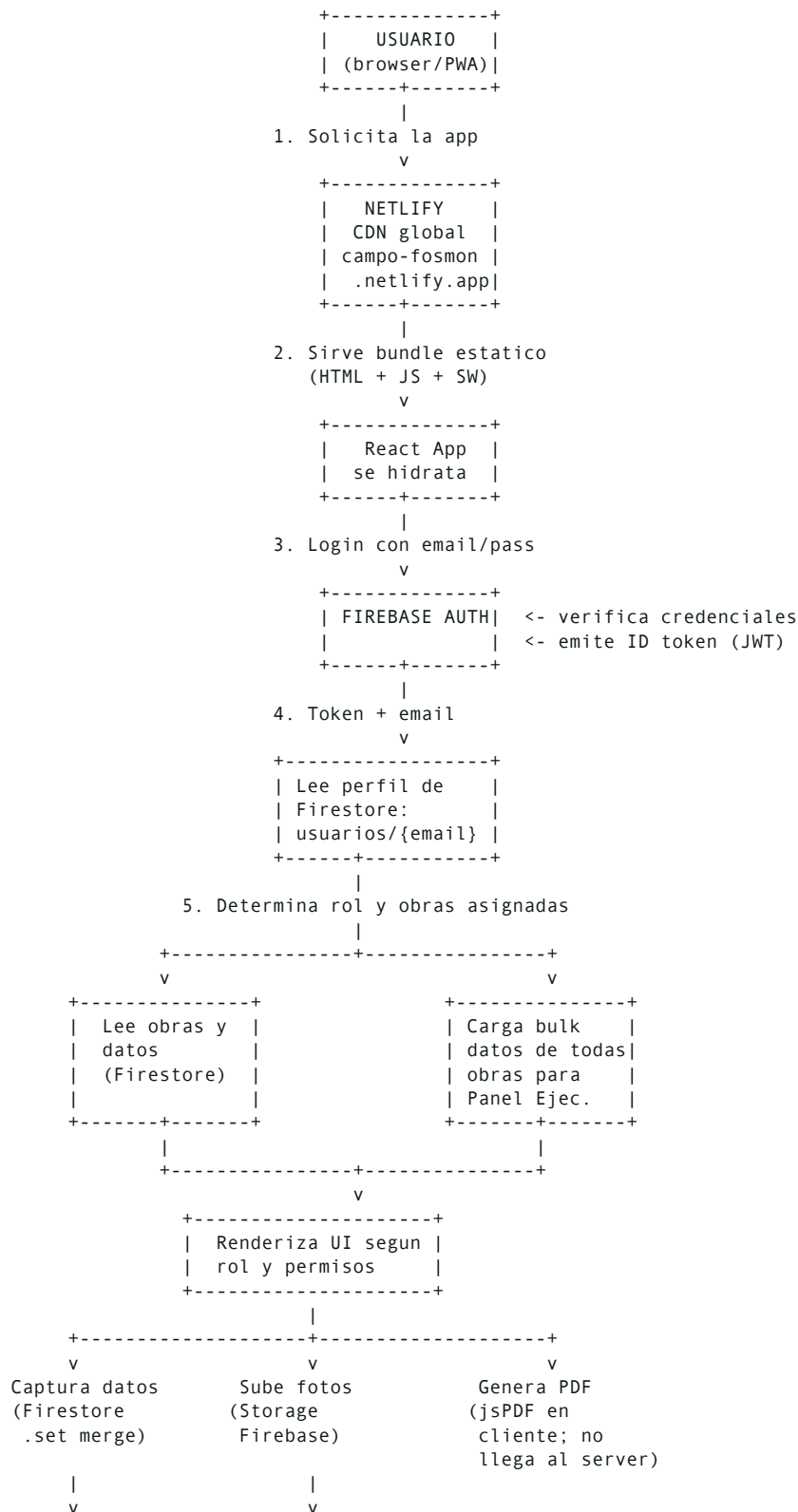
2. Stack tecnológico

Todas las piezas que conforman el sistema, de usuario a backend:



3. Diagrama de flujo detallado

Recorrido típico desde que un usuario abre CAMPO hasta que sus datos se persisten en la base. Cada flecha representa una llamada de red o una transición de estado.



FIRESTORE	STORAGE
NoSQL	Bucket

Sub-flujo: Gestión de usuarios

```

+-----+
| (solo DG / DO / admin_sistema) |
| CAMPO UI -> callable HTTPS -> |
| Cloud Functions -> Firebase Admin SDK |
| (crea / edita / elimina users en Auth |
| y perfiles en Firestore) |
+-----+

```

Sub-flujo: Integración GP Construct

```

+-----+
| Luis Mayo actualiza Google Sheet |
| cada lunes (acceso publico 'view') |
| v |
| CAMPO descarga el CSV via URL publica |
| v |
| Parser detecta obras (4 digitos), |
| rubros (3 digitos) y semanas |
| v |
| Cachea resultado en Firestore |
| (global/gp_construct) |
+-----+

```

4. Estructura de datos en Firestore

Firestore es una **base de datos NoSQL documental jerárquica** (colecciones - documentos - sub-colecciones).
Estructura del proyecto:

```
campo-fosmon (Firestore root)
|
+-- usuarios/
|   +-- {emailNormalizado}/           <- p.ej. ofosadog_fosmon_com_mx
|       fields: email, nombre, rol, obras_asignadas[], activo,
|               uid, creadoEn, creadoPor
|
+-- obras/
|   +-- {obraId}/                     <- p.ej. OAX01
|       +-- config/
|           +-- info                 <- datos generales del contrato
|           +-- parametros           <- % anticipo, FG, retencion
|           +-- estimaciones         <- array de estimaciones al cliente
|           +-- catalogo             <- catalogo de conceptos parseado
|
|       +-- avance/
|           +-- subs                 <- avance % por subseccion
|           +-- maquinaria           <- maquinaria propia
|           +-- materiales           <- materiales almacen/transito
|
|       +-- nomina/
|           +-- historial             <- semanas de nomina
|
|       +-- contrato/
|           +-- plazos               <- inicio, fin, ampliaciones
|           +-- documentos           <- lista de docs subidos
|
|       +-- subcontratos/
|           +-- lista                 <- array de subcontratos con
|                                   catalogo, pagos, fotos, adjunto
|
+-- global/
    +-- historial_obras               <- obras archivadas
    +-- gp_construct                  <- cache de Google Sheet GP
```

Decisiones de diseño

- Sin colecciones gigantes: todo está agrupado por obra para que las queries sean baratas.
- Arrays embebidos (no sub-colecciones) cuando el conjunto es pequeño/acotado (menor a 500 items): conceptos del catálogo, pagos, estimaciones.
- IDs de documento normalizados (emails con guiones bajos, obras con clave corta) para queries directas sin necesidad de índices.

5. Cloud Functions

Solo se usan Cloud Functions para lo que el frontend **no puede hacer por seguridad**: gestionar usuarios en Firebase Auth.

5 funciones HTTPS Callable (Gen 2, Node 20, región *us-central1*):

Función	Qué hace
crearUsuario	Crea en Firebase Auth + escribe perfil en Firestore
actualizarUsuario	Actualiza rol, nombre, obras_asignadas, activo
eliminarUsuario	Borra de Auth y Firestore
cambiarPassword	Admin resetea contraseña de otro usuario
listarUsuarios	Devuelve lista combinada (Firestore + estado Auth)

Guard de seguridad: todas las funciones verifican que el llamador esté autenticado y tenga rol *director_general*, *director_operaciones* o *admin_sistema* consultando su perfil en Firestore antes de ejecutar cualquier mutación.

6. Roles y permisos

Permisos definidos en código (PERMISOS y TABS_POR_ROL en App.jsx):

Rol	Ve todas las obras	Captura avance	Edita gastos	Gestiona usuarios
director_general	Sí	No	Solo ver	Sí
director_operaciones	Sí	Sí	Sí	Sí
gerente_construccion	Sí	Sí	Solo ver	No
administrador_obra	Solo asignadas	Sí	Sí	No
admin_sistema	Sí	No	Solo ver	Sí
cliente	Solo asignadas	No	No	No

El rol *cliente* tiene una UI completamente distinta: solo ve avance físico, fotos, sus estimaciones (sin desgloses internos) y plazos contractuales.

7. Seguridad

Autenticación

- Firebase Auth con email/password (no OAuth aún).
- ID tokens JWT firmados por Google. Expiración automática 1h, refresh transparente.
- Sesión persiste en localStorage del navegador (manejado por Firebase Auth).

Autorización

- **Frontend:** filtra tabs y UI por rol (map PERMISOS).
- **Backend:** Cloud Functions revalidan rol contra Firestore antes de cualquier mutación.
- **Firestore rules:** actualmente *request.auth != null* (cualquier usuario autenticado lee/escribe). Pendiente endurecer con rules por colección y rol — **deuda técnica conocida**.

Storage

- Buckets organizados por obras/{obraId}/...
- URLs de descarga firmadas por Firebase (no son públicas por path directo).

Datos sensibles

- No se almacenan datos bancarios ni de tarjetas.
- Cotizaciones y contratos firmados se guardan como archivos en Storage (acceso solo a usuarios autenticados).

8. Despliegue (CI/CD)

```
Desarrollador (Oscar/Luis) edita código local
|
v git push a rama main
|
+-----+
|  GitHub  |
| ofosado/ |
| CAMPO-FOSMON |
+-----+
| webhook  |
v
+-----+
| Netlify  | <- npm install + npm run build (Vite)
|          | <- sube bundle a CDN global
| Deploy   | <- actualiza service worker (cache busting auto)
+-----+
|
v
Live en ~2 min: campo-fosmon.netlify.app
```

Las **Cloud Functions** se despliegan aparte vía Firebase CLI (*firebase deploy --only functions*) desde la máquina del desarrollador. Tarda entre 2 y 5 minutos por función actualizada.

9. PWA (Progressive Web App)

- **Manifest:** iconos 192/512/maskable, theme color caliza #0D1619.
- **Service Worker:** generado automáticamente por vite-plugin-pwa usando Workbox.
- **Estrategia de cache:** precache de assets estáticos, CacheFirst para CDNs (jsPDF, SheetJS), NetworkFirst con fallback de 5s para Firestore.
- **Auto-update:** detecta nueva versión y la activa al cerrar/abrir la app.
- **Instalable** en iOS Safari (Compartir > Agregar a inicio), Android Chrome (banner automático) y desktop Chrome/Edge (botón en barra de URL).
- **Offline parcial:** la UI se abre sin conexión, pero la captura de datos requiere red (queda pendiente IndexedDB con cola de sync — Nivel B).

10. Integración con Google Sheets (GP Construct)

Luis es responsable de actualizar este Sheet cada lunes. CAMPO consume directamente del CSV exportado:

```
Sheet privado-con-link-publico
https://docs.google.com/spreadsheets/d/{ID}/export?format=csv
    | fetch directo (GET)
    v
CAMPO frontend
    | parser detecta obras / rubros / semanas
    v
Cache en Firestore (global/gp_construct)
    |
    v
Render en modulos de cada obra
```

Por qué no usar la API oficial de Google Sheets: evita autenticación OAuth, evita credenciales en código y simplifica permisos. El precio es que el Sheet debe quedar como *cualquiera con el enlace puede ver*.

11. Costos operativos

Servicio	Tier actual	Costo esperado
Netlify	Free	\$0/mes (límite 300 build min/mes)
Firebase Auth	Spark (incluido en Blaze)	\$0 hasta 50k MAU
Firestore	Blaze	Menos de \$1/mes con uso actual
Storage	Blaze	Menos de \$1/mes (depende de fotos)
Cloud Functions	Blaze	\$0/mes (tier 2M invocaciones gratis)
Cloud Build	Blaze	~\$0.01 por deploy de función
Dominio	Pendiente	\$0 (subdominio Netlify)

Estimado total: \$0 a \$3 USD/mes mientras estemos por debajo de los free tiers. Configurada alerta de budget en \$5 USD/mes para detectar cualquier anomalía.

12. Repositorio y arquitectura de código

- **Repo:** github.com/ofosado/CAMPO-FOSMON
- **Rama productiva:** main
- **Frontend en un solo archivo:** src/App.jsx (~6,800 líneas). Decisión deliberada: facilita ediciones rápidas en una sola pasada y evita ceremonia de imports. Cuando crezca a ~10k líneas se evaluará split.
- **Componentes principales:** Login, PantallaObras, PanelEjecutivo, Dashboard, Captura, GastosGP, Estimaciones, Riesgo, Subcontratos, Contrato, Presupuesto, GestionUsuarios, AvanceCliente, FotosCliente, EstimacionesCliente, PlazosCliente.
- **Cloud Functions:** functions/index.js (~230 líneas, JavaScript).

13. Pendientes técnicos conocidos

1. **Firestore Rules granulares** por colección y rol (hoy solo auth != null).
2. **Captura offline real** con IndexedDB y cola de sincronización.
3. **Backups automáticos de Firestore** (activar en Console).
4. **PDF de reporte** con hoja por subcontrato.
5. **Push notifications** vía Firebase Cloud Messaging.
6. **Dominio personalizado** campo.fosmon.com.mx (CNAME en DNS).
7. **Curva S** programado vs real.

14. Cómo contribuir

Para revisar el código

```
git clone https://github.com/ofosado/CAMPO-FOSMON.git
cd CAMPO-FOSMON
npm install
npm run dev    # corre local en http://localhost:5173
```

Para desplegar cambios

- **Frontend:** solo git push a main (Netlify auto-deploy).
- **Cloud Functions:** firebase deploy --only functions (requiere Firebase CLI logueado con cuenta autorizada).

Acceso a infraestructura

Servicio	URL
Firebase Console	console.firebase.google.com/project/campo-fosmon
Netlify	app.netlify.com/sites/campo-fosmon
Google Cloud Console	console.cloud.google.com/?project=campo-fosmon
Repositorio	github.com/ofosado/CAMPO-FOSMON
App productiva	campo-fosmon.netlify.app

15. Resumen ejecutivo

CAMPO es una **PWA serverless** que combina React (UI), Firebase (datos / auth / storage) y Cloud Functions (lógica privilegiada). Sin servidor propio, sin base de datos que mantener, sin contenedores. Escala automáticamente y el costo es prácticamente cero hasta que la operación crezca 10x a 100x. La complejidad está concentrada en un solo archivo para maximizar la velocidad de iteración en esta etapa temprana del producto.

Cualquier duda técnica o sugerencia de mejora, contacto directo: ofosado@fosmon.com.mx

FOSMON Construcciones · 2026